

Engenharia de Computação
Transmissão e Comunicação de Dados

RELATÓRIO DE PROJETO FINAL

Transmissão de Dados Confiável via Rádio com ESP32

Alunos:

Maurício Júnior Vieira De Sousa
Tayane Emanuely da Silva

Professor:

Ruy de Oliveira

1. Introdução

Este relatório descreve o desenvolvimento e os resultados do projeto final da disciplina de Transmissão e Comunicação de Dados. O objetivo principal foi implementar um sistema de comunicação sem fio entre dois microcontroladores ESP32 utilizando módulos de radiofrequência (RF) de 433 MHz.

O sistema desenvolvido é capaz de transmitir mensagens de texto e imagens em formato matricial binário, com detecção de erros por meio do algoritmo de checksum. Os dados transmitidos são exibidos em um display LCD I2C conectado ao receptor, permitindo a verificação visual dos resultados em tempo real.

O projeto visa consolidar os conceitos estudados em sala de aula, como estrutura de quadros, protocolos de comunicação, detecção e recuperação de erros, e transmissão de dados em camadas físicas sem fio.

2. Materiais Utilizados

- 2 placas ESP32 (microcontrolador com suporte a Wi-Fi e Bluetooth)
- 2 módulos RF 433 MHz (transmissor e receptor)
- 1 display LCD 16x2 com interface I2C (endereço 0x27)
- Cabos jumpers e protoboard
- Arduino IDE com bibliotecas: RH_ASK (RadioHead), LiquidCrystal_I2C, SPI, Wire

3. Arquitetura do Sistema

O sistema é composto por dois nós de comunicação: um transmissor e um receptor, interligados exclusivamente pelo canal de rádio de 433 MHz. Não há comunicação via Wi-Fi, Bluetooth ou qualquer outro meio — toda a troca de dados ocorre pelo módulo RF.

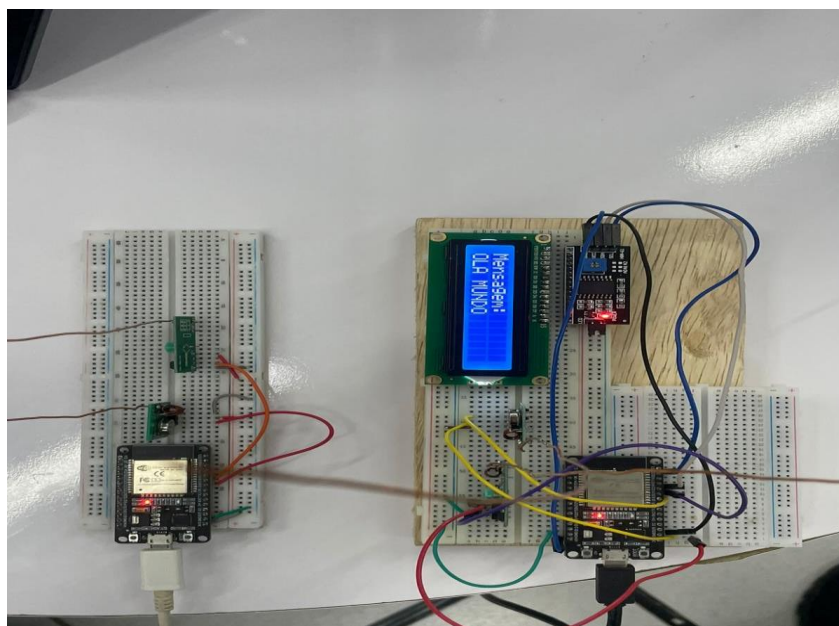


Figura 1 – Hardware montado: transmissor (esquerda) e receptor com LCD (direita)

3.1 Transmissor

O transmissor é um ESP32 conectado ao módulo RF 433 MHz pelo pino GPIO 18 (pino de dados). A biblioteca RH_ASK é configurada com velocidade de 2000 bps. O transmissor envia, em loop, primeiro uma mensagem de texto e em seguida uma imagem matricial 8x8 linha por linha.

3.2 Receptor

O receptor é um segundo ESP32 com o módulo RF 433 MHz conectado ao pino GPIO 19. Ao receber um quadro, o receptor identifica o tipo de dado (texto ou imagem) pelo prefixo do quadro, valida o checksum no caso de mensagens de texto, exibe o conteúdo no LCD e reconstrói a imagem no Monitor Serial quando todas as 8 linhas foram recebidas.

4. Estrutura de Quadros

Foram definidos dois tipos de quadros para a comunicação, identificados por um prefixo de tipo no início de cada mensagem:

Tipo	Formato	Descrição
TXT	TXT <mensagem> <checksum>	Transmissão de texto com verificação de checksum
IMG	IMG <linha> <dados>	Linha da imagem 8x8 (índice + string binária de 8 bits)

Tabela 1 – Tipos de quadros utilizados no sistema

5. Detecção de Erros – Checksum

O mecanismo de detecção de erros implementado foi o checksum por soma de valores ASCII. A função `calculaChecksum()` percorre todos os caracteres da mensagem e acumula seus valores numéricos ASCII. O resultado é inserido no quadro TXT como último campo.

No receptor, o mesmo cálculo é realizado sobre a mensagem extraída do quadro. Se o checksum recebido for igual ao calculado, a mensagem é considerada íntegra e exibida no LCD. Caso contrário, o display mostra "ERRO CHECKSUM" e o dado é descartado.

Exemplo para a mensagem "OLA MUNDO":

```
O=79, L=76, A=65, (espaço)=32, M=77, U=85, N=78, D=68, O=79
Checksum = 79+76+65+32+77+85+78+68+79 = 639
Quadro transmitido: TXT|OLA MUNDO|639
```

6. Transmissão de Imagem

A imagem transmitida é uma matriz 8x8 pixels representados em formato binário ASCII, onde '1' representa um pixel ativo e '0' um pixel inativo. A figura transmitida é um ícone circular ("smile"), codificado como:

```
Linha 0: 00111100  
Linha 1: 01000010  
Linha 2: 10100101  
Linha 3: 10000001  
Linha 4: 10100101  
Linha 5: 10011001  
Linha 6: 01000010  
Linha 7: 00111100
```

Cada linha é transmitida em um quadro IMG separado, com um intervalo de 1 segundo entre elas. O receptor armazena cada linha em um vetor e, ao receber as 8 linhas, reconstrói a imagem no Monitor Serial substituindo '1' por '#' e '0' por espaço.

7. Resultados Obtidos

Os testes realizados confirmaram o funcionamento correto do sistema nos seguintes aspectos:

- Comunicação RF: a transmissão via módulo RF 433 MHz funcionou corretamente entre as duas placas ESP32, a distâncias superiores a 1 metro.
- Exibição no LCD: mensagens de texto recebidas com checksum válido foram exibidas corretamente no display LCD 16x2, conforme ilustrado na Figura 1.
- Checksum: o mecanismo de detecção de erros operou como esperado, rejeitando quadros corrompidos e aceitando somente mensagens íntegras.
- Reconstrução de imagem: as 8 linhas da imagem foram recebidas e a figura foi reconstruída com sucesso no Monitor Serial.

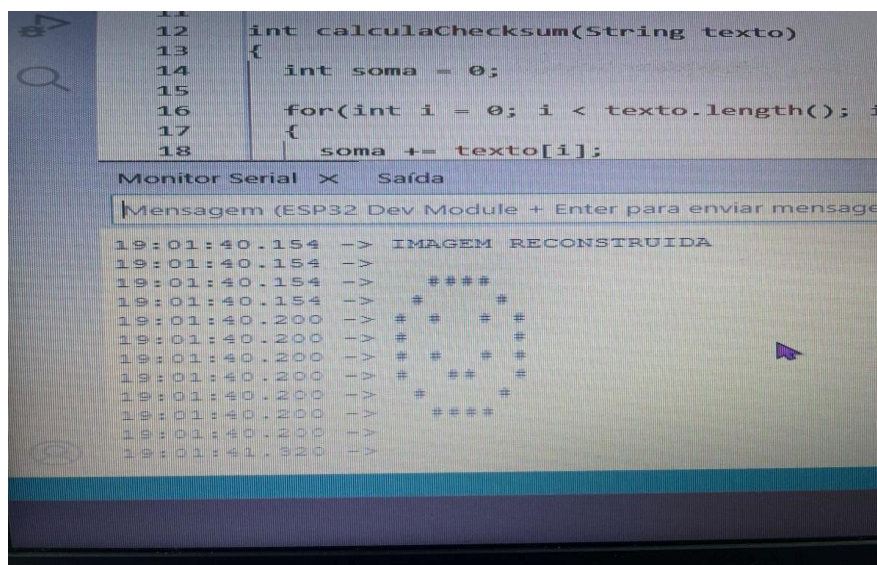


Figura 2 – Monitor Serial exibindo a imagem reconstruída no receptor

A saída do Monitor Serial comprova que todas as linhas foram recebidas e que a imagem foi corretamente montada a partir dos quadros IMG individuais, com a representação em caracteres '#' visível no terminal.

8. Limitações e Dificuldades

Durante o desenvolvimento do projeto, identificaram-se as seguintes limitações:

8.1 Protocolo Stop-and-Wait ARQ

Não foi possível implementar o protocolo Stop-and-Wait ARQ dentro do prazo do projeto. Este protocolo exigiria que o transmissor aguardasse um quadro de confirmação (ACK) do receptor antes de enviar o próximo quadro, retransmitindo em caso de timeout.

A principal dificuldade encontrada foi a natureza half-duplex dos módulos RF 433 MHz utilizados: o módulo transmissor não é capaz de receber simultaneamente, o que torna complexa a implementação de um canal de retorno para o ACK sem uma lógica de chaveamento adicional. Além disso, o tempo de sincronização e os timeouts precisariam de calibração fina para evitar colisões e retransmissões desnecessárias.

Como alternativa, o sistema opera com transmissão contínua e sem confirmação de entrega (sem ARQ), confiando no checksum para detecção de erros no lado do receptor.

8.2 Quadros ACK e END

Os tipos de quadro ACK (confirmação de recebimento) e END (fim de transmissão) previstos na especificação do projeto não foram implementados. A ausência do ACK é diretamente ligada à limitação descrita na seção anterior. O quadro END, por sua vez, não foi necessário dado o modelo de transmissão em loop adotado.

9. Código-Fonte

A seguir são apresentados os códigos-fonte completos do transmissor e do receptor implementados na Arduino IDE.

9.1 Transmissor

```
#include <RH_ASK.h>
#include <SPI.h>

RH_ASK tx(2000, -1, 18, -1);

String imagem[8] = {
  "00111100", "01000010", "10100101", "10000001",
  "10100101", "10011001", "01000010", "00111100"
};

int calculaChecksum(String texto) {
  int soma = 0;
```

```

    for(int i = 0; i < texto.length(); i++)
        soma += texto[i];
    return soma;
}

void setup() {
    Serial.begin(115200);
    if(!tx.init()) Serial.println("Erro RF");
    else Serial.println("Transmissor pronto");
    delay(2000);
}

void loop() {
    // Envio de texto
    String mensagem = "OLA MUNDO";
    int checksum = calculaChecksum(mensagem);
    String quadro = "TXT|" + mensagem + "|" + String(checksum);
    tx.send((uint8_t*)quadro.c_str(), quadro.length());
    tx.waitPacketSent();
    Serial.println("Texto enviado: " + quadro);
    delay(5000);

    // Envio da imagem (linha a linha)
    for(int i = 0; i < 8; i++) {
        String linha = "IMG|" + String(i) + "|" + imagem[i];
        tx.send((uint8_t*)linha.c_str(), linha.length());
        tx.waitPacketSent();
        delay(1000);
    }
    delay(10000);
}

```

9.2 Receptor

```

#include <RH_ASK.h>
#include <SPI.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

RH_ASK rx(2000, 19, -1, -1);
LiquidCrystal_I2C lcd(0x27, 16, 2);
String imagem[8];

int calculaChecksum(String texto) {
    int soma = 0;
    for(int i = 0; i < texto.length(); i++)
        soma += texto[i];
    return soma;
}

void setup() {
    Serial.begin(115200);
    lcd.init(); lcd.backlight();
    lcd.clear(); lcd.print("Iniciando");
    if(!rx.init()) Serial.println("Erro RF");
    else Serial.println("Receptor pronto");
}

void loop() {
    uint8_t buf[80]; uint8_t buflen = sizeof(buf);
    if(rx.recv(buf, &buflen)) {
        buf[buflen] = '\0';
        String quadro = String((char*)buf);
    }
}

```

```

if(quadro.startsWith("TXT|")) {
    int p1 = quadro.indexOf('|');
    int p2 = quadro.lastIndexOf('|');
    String mensagem = quadro.substring(p1+1, p2);
    int csRecebido = quadro.substring(p2+1).toInt();
    if(csRecebido == calculaChecksum(mensagem)) {
        lcd.clear();
        lcd.setCursor(0,0); lcd.print("Mensagem:");
        lcd.setCursor(0,1); lcd.print(mensagem);
    } else {
        lcd.clear(); lcd.print("ERRO CHECKSUM");
    }
}

if(quadro.startsWith("IMG|")) {
    int p1 = quadro.indexOf('|');
    int p2 = quadro.lastIndexOf('|');
    int linha = quadro.substring(p1+1, p2).toInt();
    imagem[linha] = quadro.substring(p2+1);
    bool completa = true;
    for(int i = 0; i < 8; i++)
        if(imagem[i] == "") completa = false;
    if(completa) {
        Serial.println("IMAGEM RECONSTRUIDA");
        for(int i = 0; i < 8; i++) {
            for(int j = 0; j < 8; j++)
                Serial.print(imagem[i][j]=='1' ? "#" : " ");
            Serial.println();
        }
        for(int i = 0; i < 8; i++) imagem[i] = "";
    }
}
}
}
}

```

10. Conclusão

O projeto demonstrou com sucesso a transmissão de dados sem fio utilizando módulos RF de 433 MHz com ESP32. Os objetivos principais foram alcançados: comunicação confiável via rádio, detecção de erros por checksum, exibição de texto no LCD e reconstrução de imagem matricial no receptor.

A principal limitação foi a não implementação do protocolo Stop-and-Wait ARQ, devido às restrições de hardware do módulo RF half-duplex e à complexidade de sincronização necessária para o canal de retorno do ACK.

O projeto proporcionou uma experiência prática relevante sobre os desafios reais de comunicação em camadas físicas sem fio, incluindo estrutura de quadros, verificação de integridade de dados e reconstrução de informação no receptor.